

Lab 4

Student: Preeti Pokharel Environment: Ubuntu Linux and WSL Date: September 28 2026

TABLE OF CONTENT

Task 1 Data Type Size Exploration

Theoretical Concept

Flow Diagram

Source Code

Compilation and Verification

Required Explanation

Summary

Task 2 Memory Segment Address Mapping

Theoretical Concept

Flow Diagram

Source Code

Compilation and Verification

Address Analysis

Summary

Knowledge Check Questions

Conclusion

List of Figures

Figure 1 Source code for data type size exploration

Figure 2 Compilation and output for data type sizes

Figure 3 Source code for observing memory segment addresses

Figure 4 Compilation and output for memory segment addresses

Figure 5 Stack and heap address difference calculation

Task 1 Data Type Size Exploration

Theoretical Concept

A data type tells the compiler and operating system how many bytes of memory a value requires, how those bytes should be interpreted, and which operations are valid. The sizeof operator reports the number of bytes allocated for a type on the current system. The result may vary with the computer architecture and operating-system data model.

Flow Diagram



Source Code

```

GNU nano 8.7.1      datatype_sizes.c
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char:      %zu byte(s)\n", sizeof(char));
    printf("int:         %zu byte(s)\n", sizeof(int));
    printf("float:       %zu byte(s)\n", sizeof(float));
    printf("double:     %zu byte(s)\n", sizeof(double));
    printf("long:       %zu byte(s)\n", sizeof(long));
    printf("unsigned int: %zu byte(s)\n", sizeof(unsigned int));
    printf("long long:  %zu byte(s)\n", sizeof(long long));

    return 0;
}
  
```

Figure 1: Source code for data type size exploration

Compilation and Verification

```
n_i_t_r_o__vpreeti@Preeti:~/Lab4$ nano datatype_sizes.c
n_i_t_r_o__vpreeti@Preeti:~/Lab4$ gcc datatype_sizes.c -o datatype_sizes
n_i_t_r_o__vpreeti@Preeti:~/Lab4$ ./datatype_sizes
=== Data Type Sizes on This System ===

char:      1 byte(s)
int:       4 byte(s)
float:     4 byte(s)
double:    8 byte(s)
long:      8 byte(s)
unsigned int: 4 byte(s)
long long:  8 byte(s)
n_i_t_r_o__vpreeti@Preeti:~/Lab4$ |
```

Figure 2: Compilation and output for data type sizes

Required Explanation

The size of long may differ because operating systems use different data models. On many 32-bit systems, long occupies 4 bytes, while on 64-bit Linux systems it normally occupies 8 bytes so that it can represent a larger range of integer values.

Summary

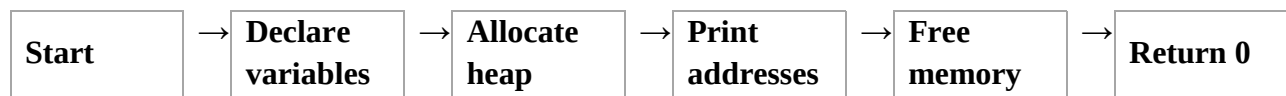
The program compiled without errors and displayed the expected sizes for all required data types. The results matched the typical values for a 64-bit Linux system.

Task 2 Memory Segment Address Mapping

Theoretical Concept

A process uses separate memory regions for different kinds of information. Initialized global and static variables are stored in the Data segment, uninitialized global and static variables are stored in the BSS segment, local variables are stored on the Stack, and dynamically allocated values are stored in the Heap. Printing their addresses demonstrates this virtual memory organization.

Flow Diagram



Source Code

```

GNU nano 8.7.1 memory_segments.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 150; // Data segment
int bss_var;          // BSS/Data segment

int main() {
    int local_var = 30; // Stack
    static int local_static = 20; // Data segment

    int *heap_var = (int *)malloc(sizeof(int)); // Heap
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");

    printf("global_var:  %p (Data/BSS)\n",
           (void *)&global_var);

    printf("bss_var:      %p (Data/BSS)\n",
           (void *)&bss_var);

    printf("local_var:     %p (Stack)\n",
           (void *)&local_var);

    printf("local_static: %p (Data/BSS)\n",
           (void *)&local_static);

    printf("heap_var ptr: %p (Stack - pointer)\n",
           (void *)&heap_var);

    printf("*heap_var:   %p (Heap - data)\n",
           (void *)heap_var);

    free(heap_var);

    return 0;
}
  
```

Figure 3: Source code for observing memory segment addresses

Task 2 Memory Segment Address Mapping Continued

Compilation and Verification

```
n_i_t_r_o__vpreeti@Preeti:~/Lab4$ nano memory_segments.c
n_i_t_r_o__vpreeti@Preeti:~/Lab4$ gcc memory_segments.c -o memory_segments
n_i_t_r_o__vpreeti@Preeti:~/Lab4$ ./memory_segments
.....Variable Addresses.....

global_var: 0x6490cd10a010 (Data/BSS)
bss_var: 0x6490cd10a01c (Data/BSS)
local_var: 0x7ffccf874b0c (Stack)
local_static: 0x6490cd10a014 (Data/BSS)
heap_var ptr: 0x7ffccf874b10 (Stack - pointer)
*heap_var: 0x6490d2a46010 (Heap - data)
n_i_t_r_o__vpreeti@Preeti:~/Lab4$ |
```

Figure 4: Compilation and output for memory segment addresses

Address Analysis

The Stack had the highest observed address at 0x7ffccf874b0c. Among the displayed variables, the initialized global variable in the Data segment had the lowest address at 0x6490cd10a010. The dynamically allocated value was stored in the Heap at 0x6490d2a46010.

```
n_i_t_r_o__vpreeti@Preeti:~/Lab4$ echo $((0x7ffccf874b0c - 0x6490d2a46010))
30150618180348
n_i_t_r_o__vpreeti@Preeti:~/Lab4$ |
```

Figure 5: Stack and heap address difference calculation

The absolute difference between the observed Stack and Heap addresses was 30,150,618,180,348 bytes. The exact addresses may change each time the program runs because modern operating systems use address space layout randomization.

Summary

The output confirmed that local variables and pointer variables occupied high Stack addresses, dynamic memory occupied the Heap, and global or static variables occupied lower Data and BSS addresses. The program therefore demonstrated the expected organization of a process virtual address space.

Knowledge Check Questions

Question 1 Meaning of a data type

A data type is a contract that specifies how much memory must be allocated, how the stored bytes should be interpreted, and which operations are valid for the value.

Question 2 Five main memory segments

The five main process memory segments are the Text segment, Data segment, BSS segment, Heap segment, and Stack segment.

\

Question 3 Difference between Data and BSS

The Data segment stores initialized global and static variables. The BSS segment stores uninitialized or zero-initialized global and static variables.

Question 4 High Stack and low Text addresses

The Stack normally begins at high virtual addresses and grows downward, while the Heap grows upward. Keeping the Stack high and the Text segment low separates instructions from temporary data and leaves room for dynamic growth.

Question 5 Stack memory after a function returns

When a function returns, its stack frame is automatically released by adjusting the stack pointer. The compiled program and runtime perform this operation automatically, so the programmer does not call free for stack memory.

Question 6 How the OS identifies process memory

The operating system gives each process a separate virtual address space and maintains its page tables. The memory management unit uses those page tables to map virtual addresses to physical memory while keeping processes isolated.

Conclusion

This lab demonstrated the relationship between C data types and operating-system memory management. The first task measured the storage required by common data types on a 64-bit Linux system. The second task showed that global, static, local, and dynamically allocated variables occupy different memory segments and confirmed that the Stack uses the highest observed virtual addresses.